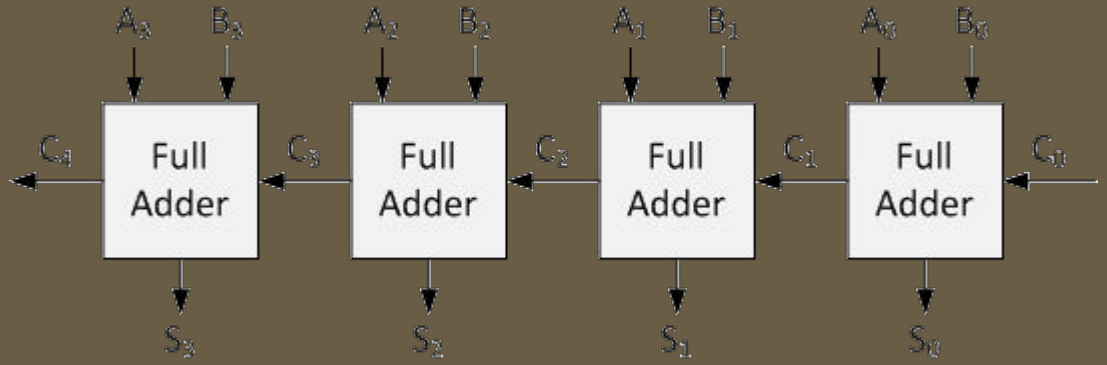
4-bit Carry Adder



Name: David (王琮文)
Class: Digital Design with FPGA
Date: Nov. 03, 2025

國立陽明交通大學

陽明交大雷射系統研究中心

4-bit Carry Adder

1. Fundamentals

Start with gate-level building blocks and compare RCA and CLA performance.

2. Toolchain

Install and practice with Quartus Prime Lite and ModelSim to run examples.

3. Coding & verification

Implement designs in VHDL and verify them with testbenches in simulation.

4. Hardware & validation

Program the DE2-115 board and validate that hardware behaviour matches ModelSim.

Roadmap

Step 1. Concepts

- half adders
- full adders
- ripple-carry adder (RCA)
- carry-lookahead adder (CLA)

Step 2. Simulation

- Install Quartus Prime
- Install ModelSim
- Write VHDL scripts
- Implement RCA and CLA
- Develop block designs
- Write Testbench
- Simulate in ModelSim

Step 3. Experiments

- Design 7-seg display
- Program DE2-115
- Compare simulation and experiments

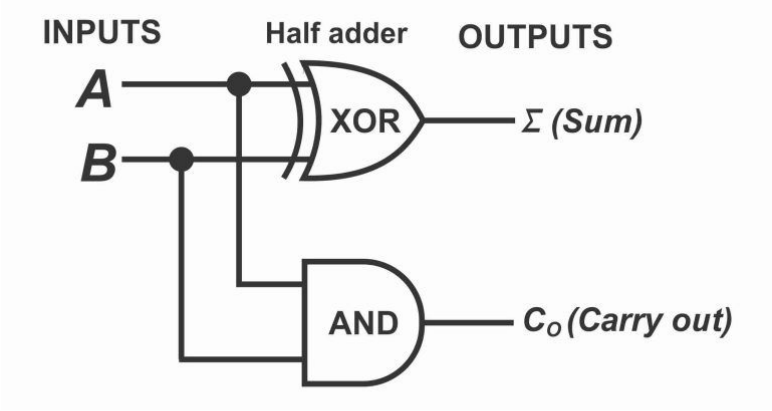
Fundamentals

Adder Types

- ➔ half adder
- ➔ full adder
- ➔ ripple-carry adder (RCA)
- ➔ carry-lookahead adder (CLA)

Half Adder

- A half adder is a digital circuit that adds two single binary digits (bits) and produces a two-bit sum: a sum bit and a carry bit.
- Logic Gates: one XOR gate (for the sum) and one AND gate (for the carry).
- Sum (S): The result of an XOR gate
  S = A ⊕ B
- Carry (C): The result of an AND gate
  C = A • B
- A key limitation is that it cannot account for a carry-in bit from a previous addition, which is handled by a full adder.

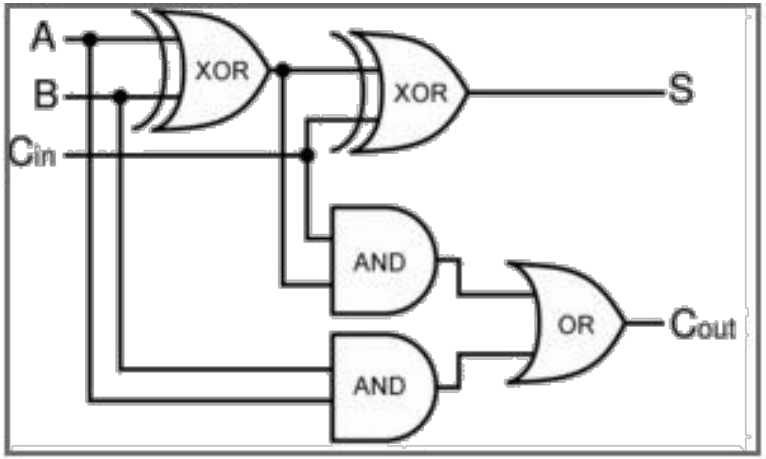


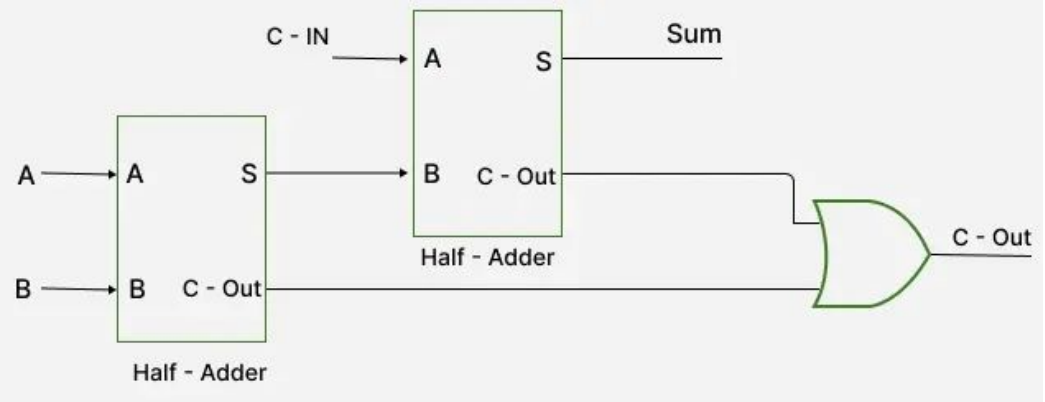
Truth Table

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

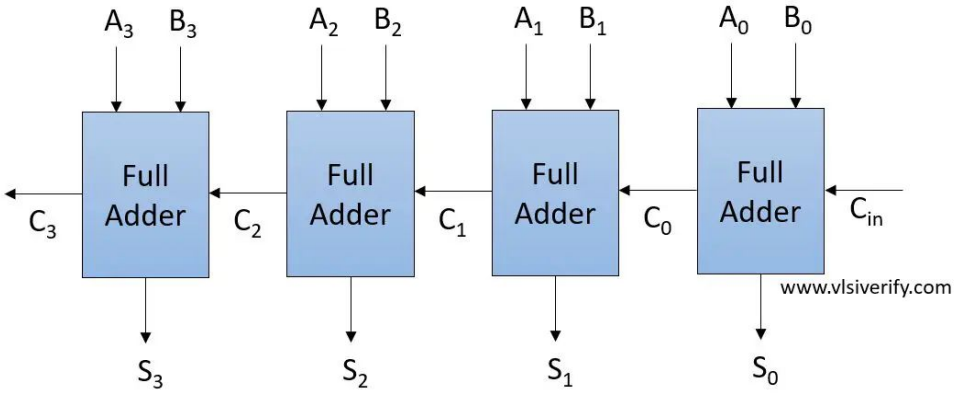
Full Adder

- A full adder is a digital circuit that adds three single-bit binary numbers and produces a two-bit output: a sum and a carry-out.
- Sum
  S = A ⊕ B ⊕ Cin
- Carry-out
  C = AB + ACin + BCin
  = AB + Cin (A ⊕ B)



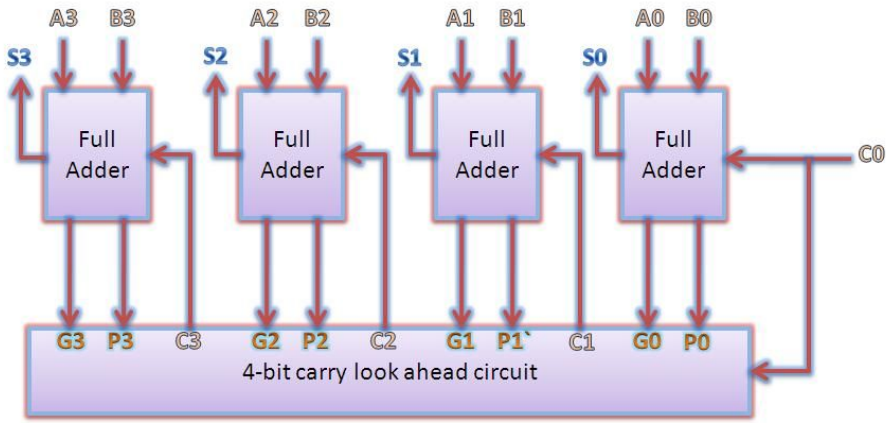


Ripple-Carry Adder (RCA)



- A ripple-carry adder is a digital circuit that adds two binary numbers by **cascading** multiple **full adder** circuits.
- The carry-out of the first full adder is connected to the carry-in of the second, and so on. The carry "ripples" from the least significant bit to the most significant bit.
- **Propagation delay**: Each full adder must wait for the carry-in from the previous stage, so a significant propagation delay increases with the number of bits.

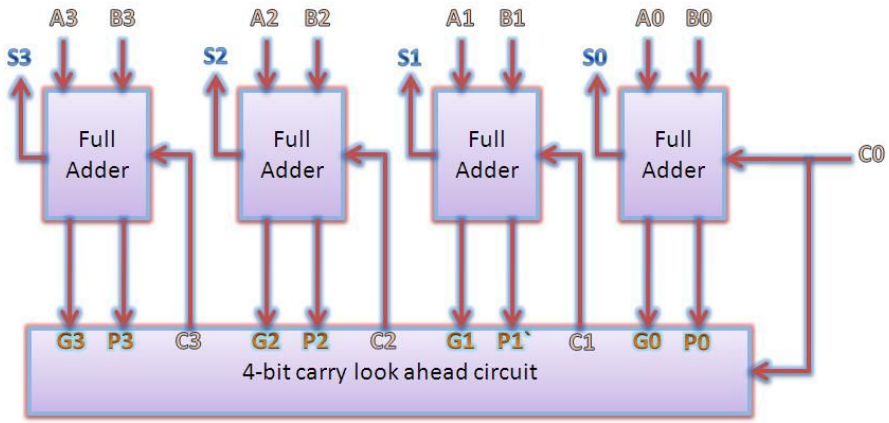
Carry-Lookahead Adder (CLA)



- A CLA uses separate logic to calculate carries in advance. It does this by creating two signals (G and P) for each bit.
- Generate G : A carry is generated if both input bits are one ($Gi = Ai \cdot Bi$).
- Propagate P : A carry is propagated if at least one of the input bits is one and the carry-in is one ($Pi = Ai \oplus Bi$).
- Benefits: much faster carry determination for wide adders; useful in high-speed ALUs and CPUs.
- **Trade-offs**: more hardware (extra gates/wires) and more complex wiring compared with simple ripple-carry adders.

Carry-Lookahead Adder (CLA)

Example: Assume inputs are A = 1011, B = 0110, Cin = 0.



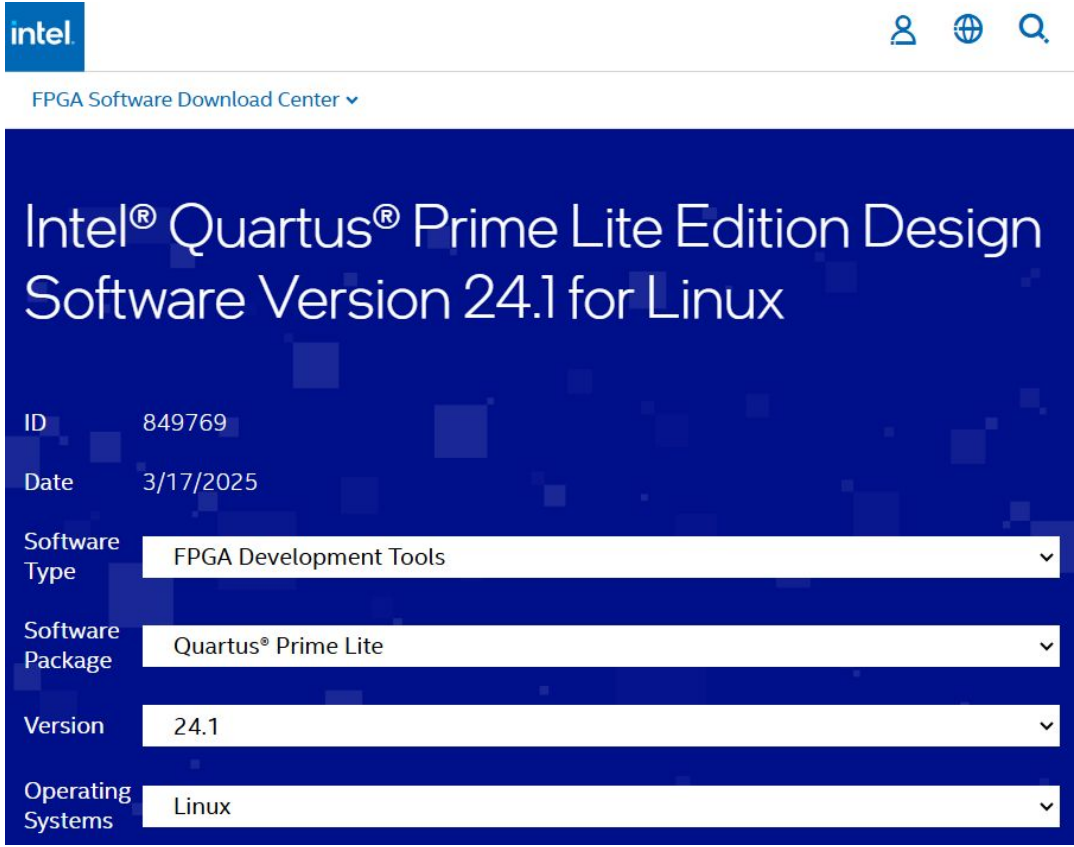
- A = $a_3a_2a_1a_0$ = 1011
- B = $b_3b_2b_1b_0$ = 0110
- Compute g_i and p_i for each bit:
  - $g_0 = a_0 \cdot b_0 = 1 \cdot 0 = 0,\ p_0 = a_0 \oplus b_0 = 1 \oplus 0 = 1$
  - $g_1 = 1 \cdot 1 = 1,\ p_1 = 1 \oplus 1 = 0$
  - $g_2 = 0 \cdot 1 = 0,\ p_2 = 0 \oplus 1 = 1$
  - $g_3 = 1 \cdot 0 = 0,\ p_3 = 1 \oplus 0 = 1$
- Compute carries using CLA formulas with $c_0 = 0$:
  - $c_1 = g_0 + p_0 \cdot c_0 = 0 + 1 \cdot 0 = 0$
  - $c_2 = g_1 + p_1g_0 + p_1p_0c_0 = 1 + 0 \cdot 0 + 0 \cdot 1 \cdot 0 = 1$
  - $c_3 = g_2 + p_2g_1 + p_2p_1g_0 + p_2p_1p_0c_0 = 0 + 1 \cdot 1 + 1 \cdot 0 \cdot 0 + 1 \cdot 0 \cdot 1 \cdot 0 = 1$
  - $c_4 = g_3 + p_3g_2 + p_3p_2g_1 + p_3p_2p_1g_0 + p_3p_2p_1p_0c_0 = 0 + 1 \cdot 0 + 1 \cdot 1 \cdot 1 + \cdots = 1$
- Compute sums:
  - $s_0 = p_0 \oplus c_0 = 1 \oplus 0 = 1$
  - $s_1 = p_1 \oplus c_1 = 0 \oplus 0 = 0$
  - $s_2 = p_2 \oplus c_2 = 1 \oplus 1 = 0$
  - $s_3 = p_3 \oplus c_3 = 1 \oplus 1 = 0$
- Sum bits $s_3s_2s_1s_0$ = 0 0 0 1 with final carry $c_4 = 1$, so the full result is 1 0001 (binary) = 17 (decimal). The CLA produced carries and sums using the generate/propagate logic rather than waiting bit-by-bit propagation

Toolchain

Installation & Practice

- ➔ Quartus Prime Lite
- ➔ ModelSim
- ➔ VHDL Coding
- ➔ Block Design

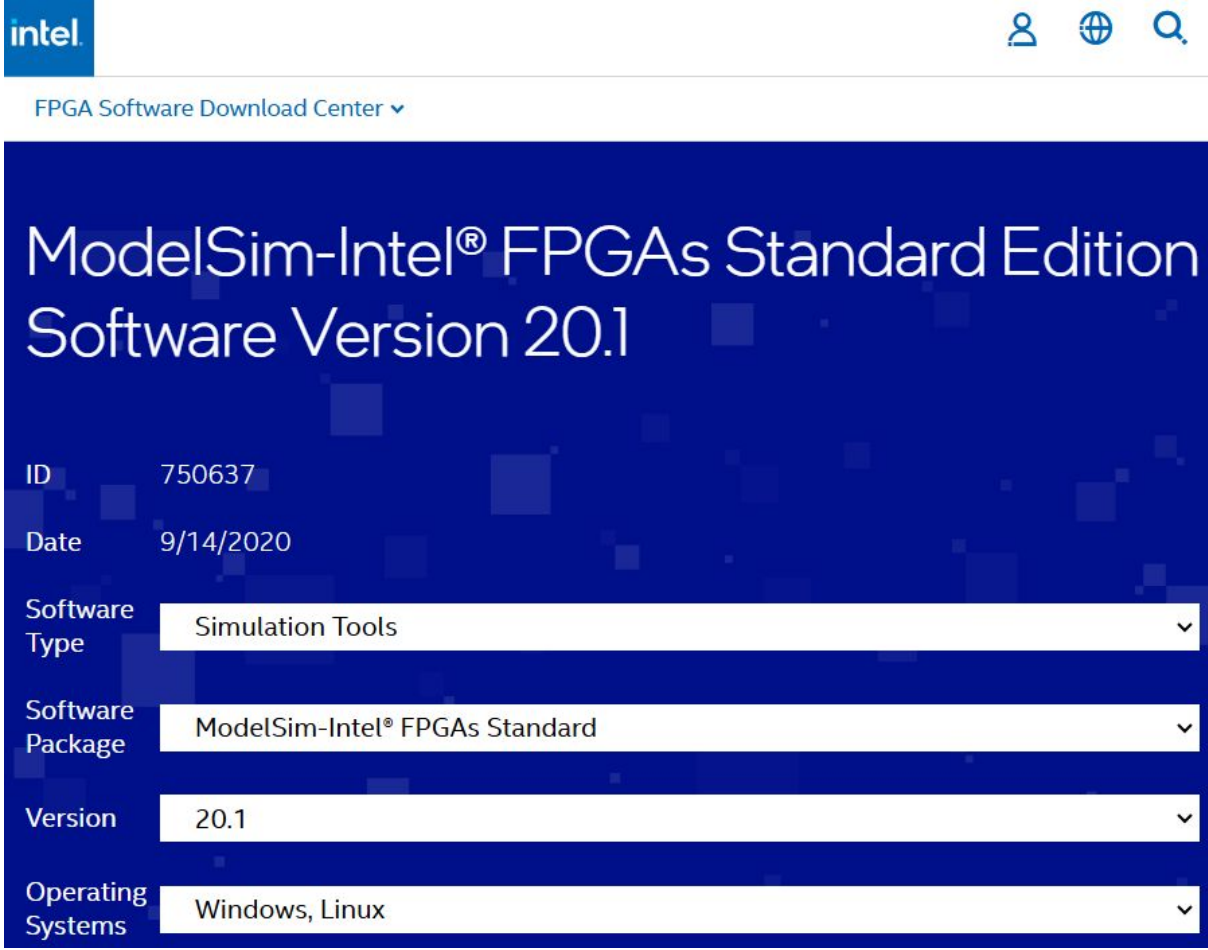
Install Quartus Prime Lite



- Source

https://www.intel.com/content/www/us/en/software-kit/849769/intel-quartus-prime-lite-edition-design-software-version-24-1-for-linux.html

Install ModelSim



- Source
  https://www.intel.com/content/www/us/en/software-kit/750637/modelsim-intel-fpgas-standard-edition-software-version-20-1.html

VHDL Basics

- VHDL, which stands for **Very High-Speed Integrated Circuit Hardware Description Language**, is a programming language used to describe the behavior and structure of digital electronic circuits. It allows engineers to model, simulate, and synthesize digital systems, from simple logic gates to complex integrated circuits, for use in applications like FPGAs and ASICs. VHDL handles both concurrent and sequential operations and incorporates timing specifications, and it is standardized by the IEEE.

- NandLand Tutorial – Introduction to VHDL
  - https://nandland.com/introduction-to-vhdl-for-beginners-with-code-examples/

Design Entry in Quartus

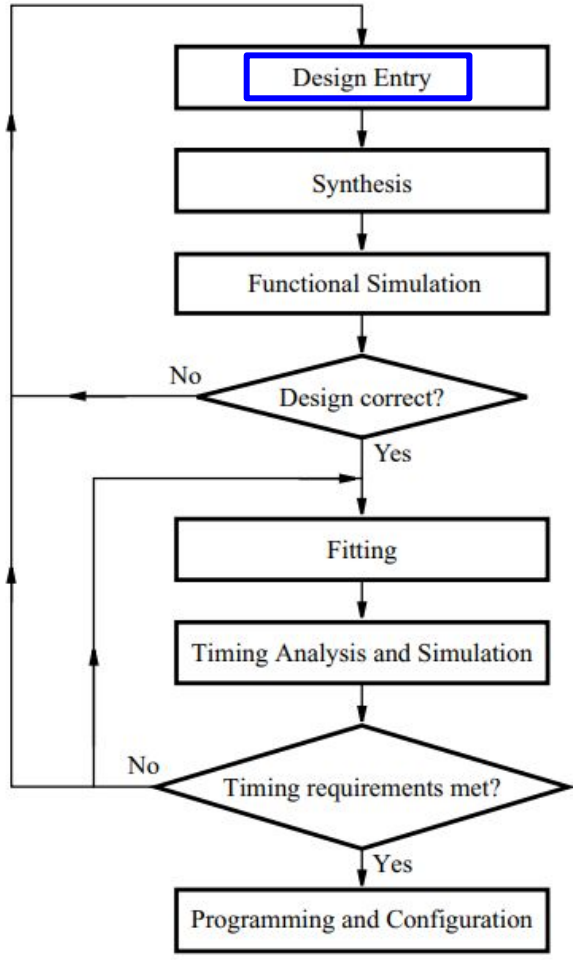


Figure 1. Typical CAD flow.

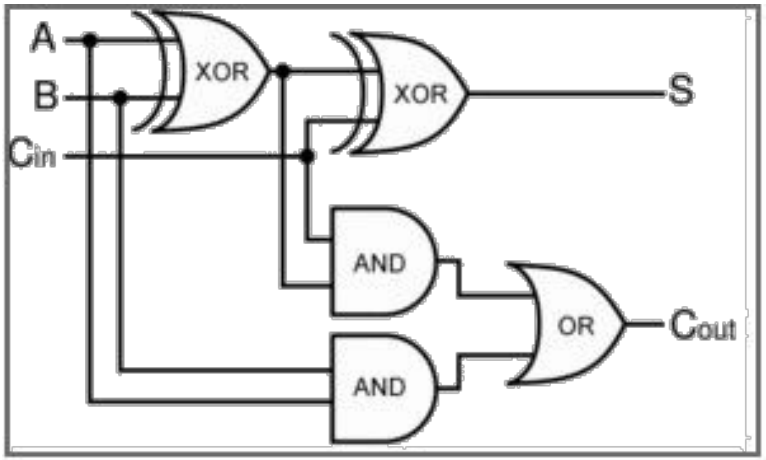
The desired circuit is specified either by means of a **schematic diagram**, or by using a **hardware description language**, such as Verilog or VHDL.

Coding & Verification

Simulation

- Implement full adders with VHDL
- Implement ripple-carry adder (RCA) with VHDL
- Implement carry-lookahead adder (CLA) with VHDL
- Develop schematic designs with I/O constraints
- Write and compile testbenches on ModelSim

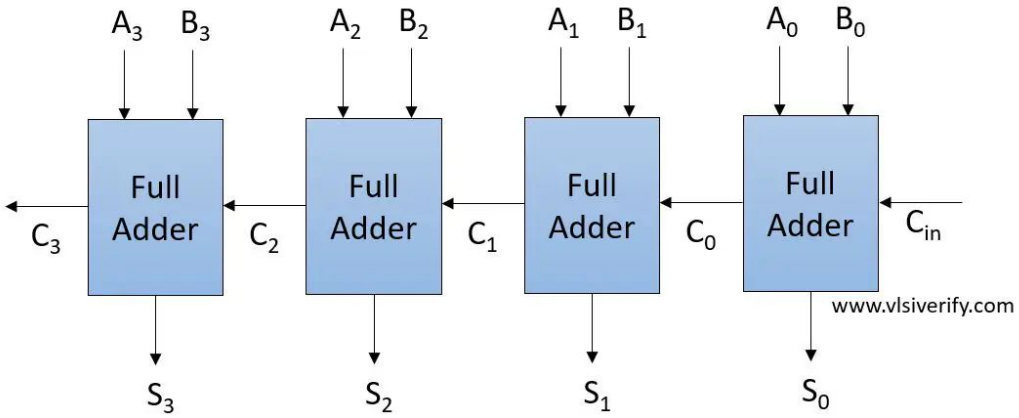
Implement a full adder with VHDL




```vhdl
entity full_adder is
  port (
    a   : in  std_logic;
    b   : in  std_logic;
    cin : in  std_logic;
    s   : out std_logic;
    cout: out std_logic
  );
end entity;

architecture rtl of full_adder is
begin
  s    <= a xor b xor cin;
  cout <= (a and b) or (a and cin) or (b and cin);
end architecture;
```

Implement an RCA with VHDL

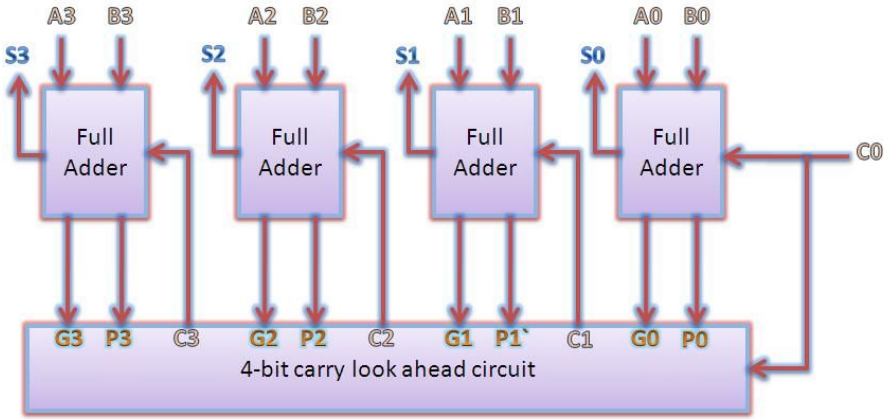
ripple-carry adder (RCA) = 4 full adders




```vhdl
entity rca_4bit is
  port(
    a     : in  std_logic_vector(3 downto 0);
    b     : in  std_logic_vector(3 downto 0);
    cin   : in  std_logic;
    sum   : out std_logic_vector(3 downto 0);
    cout  : out std_logic
  );
end entity;

architecture structural of rca_4bit is
  signal c : std_logic_vector(4 downto 0);
begin
  c(0) <= cin;

  fa0: entity work.full_adder(rtl)
    port map(a(0), b(0), c(0), sum(0), c(1));

  fa1: entity work.full_adder(rtl)
    port map(a(1), b(1), c(1), sum(1), c(2));

  fa2: entity work.full_adder(rtl)
    port map(a(2), b(2), c(2), sum(2), c(3));

  fa3: entity work.full_adder(rtl)
    port map(a(3), b(3), c(3), sum(3), c(4));

  cout <= c(4);
end architecture;
```

Implement an CLA with VHDL

carry-lookahead adder (CLA)



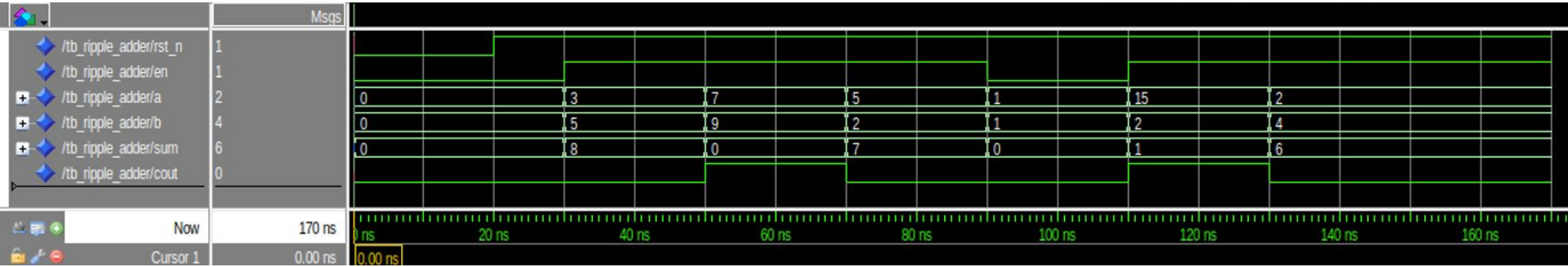

```vhdl
architecture rtl of cla_4bit is
  -- propagate and generate for each bit:
  -- p(i) = a(i) xor b(i)  -> indicates this bit will propagate an incoming carry
  -- g(i) = a(i) and b(i)  -> indicates this bit will generate a carry regardless of incoming carry
  signal p, g : std_logic_vector(3 downto 0);

  -- carries: c(0) is input carry (cin), c(4) is output carry (cout)
  signal c    : std_logic_vector(4 downto 0);
begin
  -- assign initial carry
  c(0) <= cin;

  -- generate p and g for each bit 0..3
  gen_pg: for i in 0 to 3 generate
    p(i) <= a(i) xor b(i);      -- propagate
    g(i) <= a(i) and b(i);      -- generate
  end generate gen_pg;

  -- carry equations expanded (lookahead expansion for clarity)
  -- c(1) is carry into bit 1 (i.e., carry out from bit 0)
  c(1) <= g(0) or (p(0) and c(0));

  -- c(2) is carry into bit 2 (carry out from bit 1)
  c(2) <= g(1) or (p(1) and g(0)) or (p(1) and p(0) and c(0));

  -- c(3) is carry into bit 3 (carry out from bit 2)
  c(3) <= g(2) or (p(2) and g(1)) or (p(2) and p(1) and g(0)) or (p(2) and p(1) and p(0) and c(0));

  -- c(4) is final carry out (carry out from bit 3)
  c(4) <= g(3)
          or (p(3) and g(2))
          or (p(3) and p(2) and g(1))
          or (p(3) and p(2) and p(1) and g(0))
          or (p(3) and p(2) and p(1) and p(0) and c(0));

  -- sum bits: sum(i) = p(i) xor c(i)  (where c(i) is carry into bit i)
  sum(0) <= p(0) xor c(0);
  sum(1) <= p(1) xor c(1);
  sum(2) <= p(2) xor c(2);
  sum(3) <= p(3) xor c(3);

  -- final carry out
  cout <= c(4);

end rtl;
```

Test Cases

- Test Case #1: enabled, 3 + 5 → ideal sum = 8
- Test Case #2: enabled, 7 + 9 → ideal carry-out will be one
- Test Case #3: enabled, 5 + 2 → ideal sum = 7
- Test Case #4: disabled, 1 + 1 → outputs = previous value (sum = 7)
- Test Case #5: enabled, 15 + 2 → (sum, cout) = (1, 1)
- Test Case #6: enabled, 2 + 4 → ideal sum = 6

Start compilation and see RTL simulation

ModelSim®



Hardware & Validation

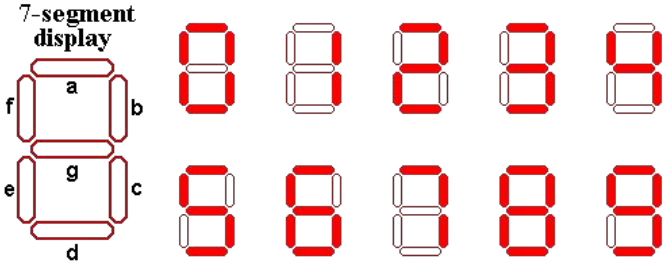
Experiments with DE2-115

- ➔ 7-Segment Display
- ➔ Block Designs
- ➔ I/O Pins
- ➔ DE2-115 Programming

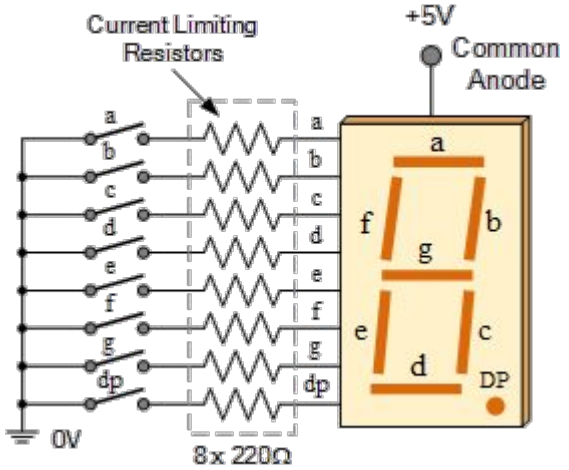
Altera DE2-115

The Altera DE2-115 is an FPGA development and education kit that features the Cyclone IV EP4CE115 FPGA, which offers 114,480 logic elements (LEs), up to 3.9 Mbits of RAM, and 266 multipliers.

7-Segment Display

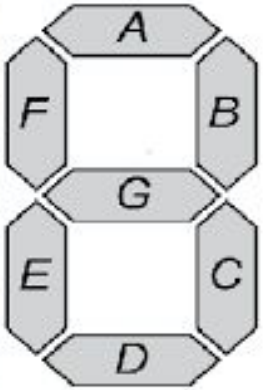


A 7-segment display consists of seven LEDs arranged in a figure-eight pattern. Each segment is labeled A through G, and when illuminated in specific combinations, they form digits (0–9) and letters (A–F).



7-Segment Display on DE2-115

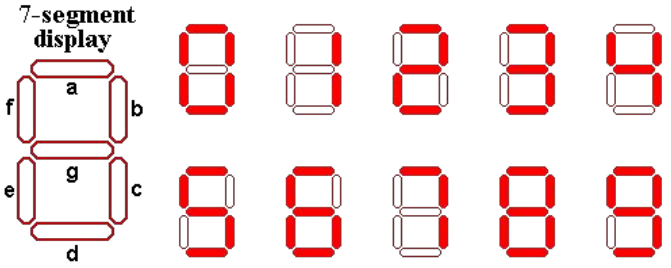
Digit	Display	Input	A	B	C	D	E	F	G
0	0	0000	on	on	on	on	on	on	off
1	1	0001	off	on	on	off	off	off	off
2	2	0010	on	on	off	on	on	off	on
3	3	0011	on	on	on	on	off	off	on
4	4	0100	off	on	on	off	off	on	on
5	5	0101	on	off	on	on	off	on	on
6	6	0110	on	off	on	on	on	on	on
7	7	0111	on	on	on	off	off	off	off
8	8	0111	on	on	on	on	on	on	on
9	9	1001	on	on	on	on	off	on	on




```
58        with digit select
59          seg_pat <=
60            "0000001" when "0000", -- 0: a b c d e f ON, g OFF
61            "1001111" when "0001", -- 1
62            "0010010" when "0010", -- 2
63            "0000110" when "0011", -- 3
64            "1001100" when "0100", -- 4
65            "0100100" when "0101", -- 5
66            "0100000" when "0110", -- 6
67            "0001111" when "0111", -- 7
68            "0000000" when "1000", -- 8: all segments ON
69            "0000100" when "1001", -- 9
70            "0000001" when "1010", -- 10 -> same pattern as 0 (adjust if you want 'A')
71            "1001111" when "1011", -- 11 -> same as 1
72            "0010010" when "1100", -- 12 -> same as 2
73            "0000110" when "1101", -- 13 -> same as 3
74            "1001100" when "1110", -- 14 -> same as 4
75            "0100100" when "1111", -- 15 -> same as 5
76            "1111111" when others;  -- default: all segments OFF (blank)
```

Display CLA Results on DE2-115

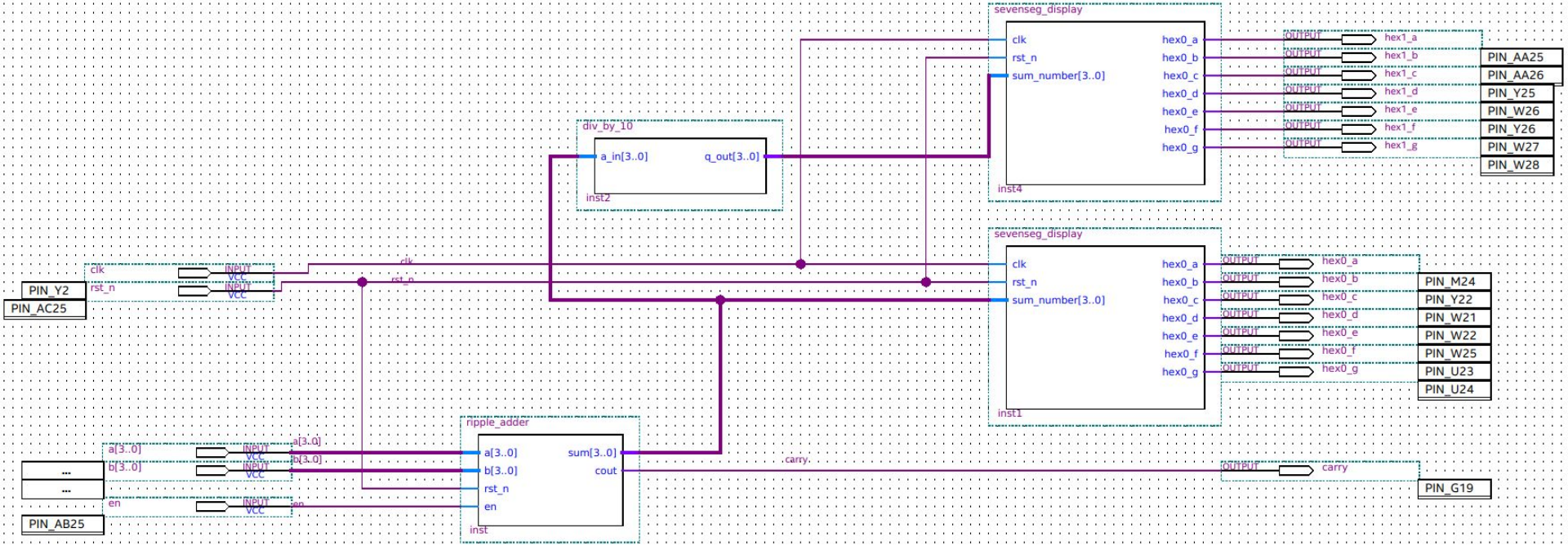
(CLA = carry-lookahead adder)



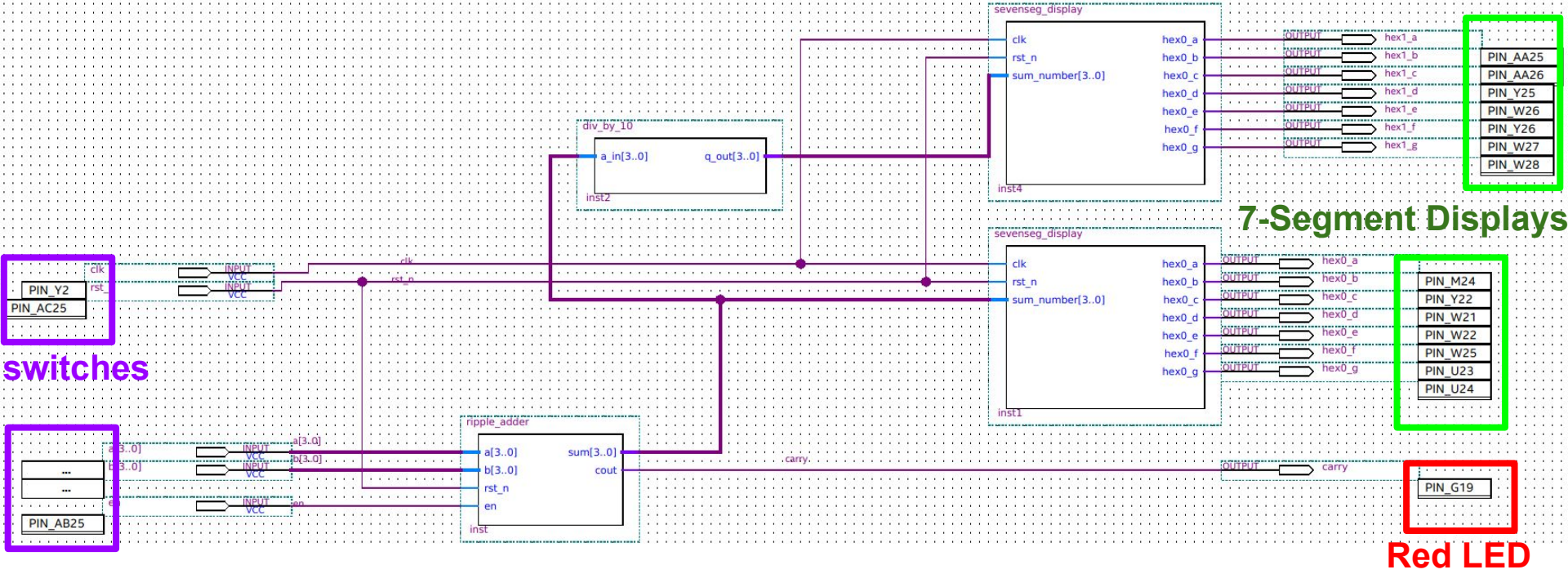
Assume rst_n = ‘1’ and en = ‘1’.

[Output Case] (decimal) summation of 2 numbers	(binary) sum in VHDL	7-segment display	carry-out in VHDL	LED status
6	0110	06	0	OFF
8	1000	08	0	OFF
10	1010	10	0	OFF
15	1111	15	0	OFF
16	0000	00	1	ON
17	0001	01	1	ON

Block Design



Block Design: I/O Pins



Pin Assignment to DE2-115

Check the user manual and refer to the pins like SW[•], LEDR[•], and HEX1[•] for switches, LEDs, and 7-segment displays, respectively.

Table 4-1 Pin Assignments for Slide Switches

Signal Name	*FPGA Pin No.*	*Description*	*I/O Standard*
SW[0]	PIN_AB28	Slide Switch[0]	Depending on JP7
SW[1]	PIN_AC28	Slide Switch[1]	Depending on JP7
SW[2]	PIN_AC27	Slide Switch[2]	Depending on JP7
SW[3]	PIN_AD27	Slide Switch[3]	Depending on JP7

Table 4-3 Pin Assignments for LEDs

Signal Name	*FPGA Pin No.*	*Description*	*I/O Standard*
LEDR[0]	PIN_G19	LED Red[0]	2.5V
LEDR[1]	PIN_F19	LED Red[1]	2.5V
LEDR[2]	PIN_E19	LED Red[2]	2.5V

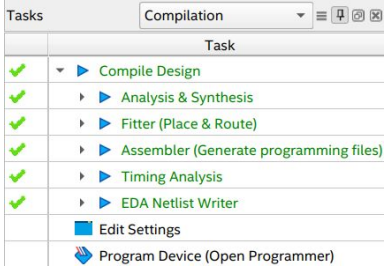
Table 4-4 Pin Assignments for 7-segment Displays

Signal Name	*FPGA Pin No.*	*Description*	*I/O Standard*
HEX0[0]	PIN_G18	Seven Segment Digit 0[0]	2.5V
HEX0[1]	PIN_F22	Seven Segment Digit 0[1]	2.5V
HEX0[2]	PIN_E17	Seven Segment Digit 0[2]	2.5V

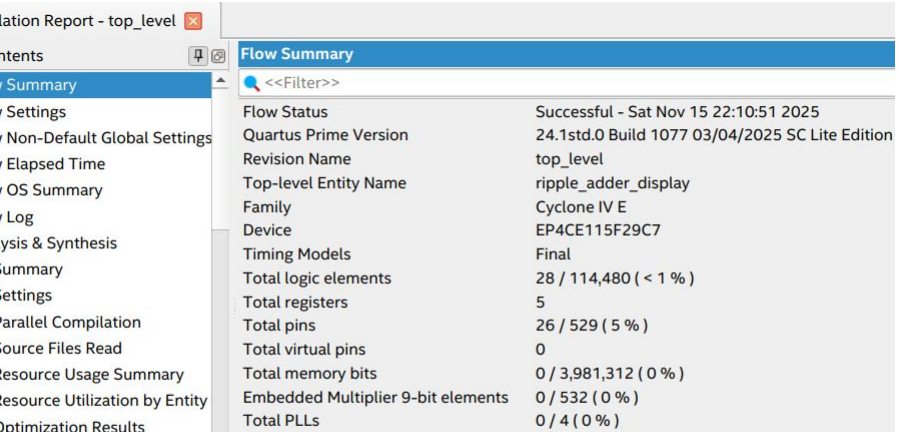
	Status	From	To	Assignment Name	Value	Enabled	Entity
1	Ok		clk	Location	PIN_Y2	Yes	
2	Ok		clk	I/O Standard	3.3-V LVTTL	Yes	ripple_...display
3	Ok		rst_n	Location	PIN_AC25	Yes	
4	Ok		a[0]	Location	PIN_AB28	Yes	
5	Ok		a[1]	Location	PIN_AC28	Yes	
6	Ok		a[2]	Location	PIN_AC27	Yes	
7	Ok		a[3]	Location	PIN_AD27	Yes	
8	Ok		b[0]	Location	PIN_AB27	Yes	
9	Ok		b[1]	Location	PIN_AC26	Yes	
10	Ok		b[2]	Location	PIN_AD26	Yes	
11	Ok		b[3]	Location	PIN_AB26	Yes	
12	Ok		en	Location	PIN_AB25	Yes	
13	Ok		carry	Location	PIN_G19	Yes	
14	Ok		hex0_a	Location	PIN_M24	Yes	
15	Ok		hex0_b	Location	PIN_Y22	Yes	
16	Ok		hex0_c	Location	PIN_W21	Yes	
17	Ok		hex0_d	Location	PIN_W22	Yes	
18	Ok		hex0_e	Location	PIN_W25	Yes	
19	Ok		hex0_f	Location	PIN_U23	Yes	
20	Ok		hex0_g	Location	PIN_U24	Yes	
21	Ok		hex0_a	I/O Standard	3.3-V LVTTL	Yes	ripple_...display
22	Ok		hex0_b	I/O Standard	3.3-V LVTTL	Yes	ripple_...display
23	Ok		hex0_c	I/O Standard	3.3-V LVTTL	Yes	ripple_...display
24	Ok		hex0_d	I/O Standard	3.3-V LVTTL	Yes	ripple_...display
25	Ok		hex0_e	I/O Standard	3.3-V LVTTL	Yes	ripple_...display

User Manual:
https://www.terasic.com.tw/attachment/archive/502/DE2_115_User_manual.pdf

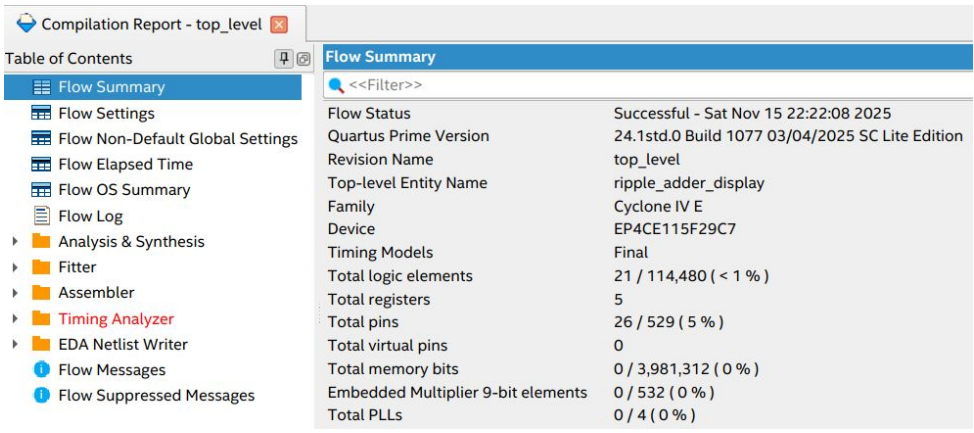
After compilation …



carry-lookahead adder (CLA)



ripple-carry adder (RCA)



total logic elements = 28

total logic elements = 21

Resource Usage

carry-lookahead adder (CLA) > ripple-carry adder (RCA)

Analysis & Synthesis Resource Usage Summary

<<Filter>>

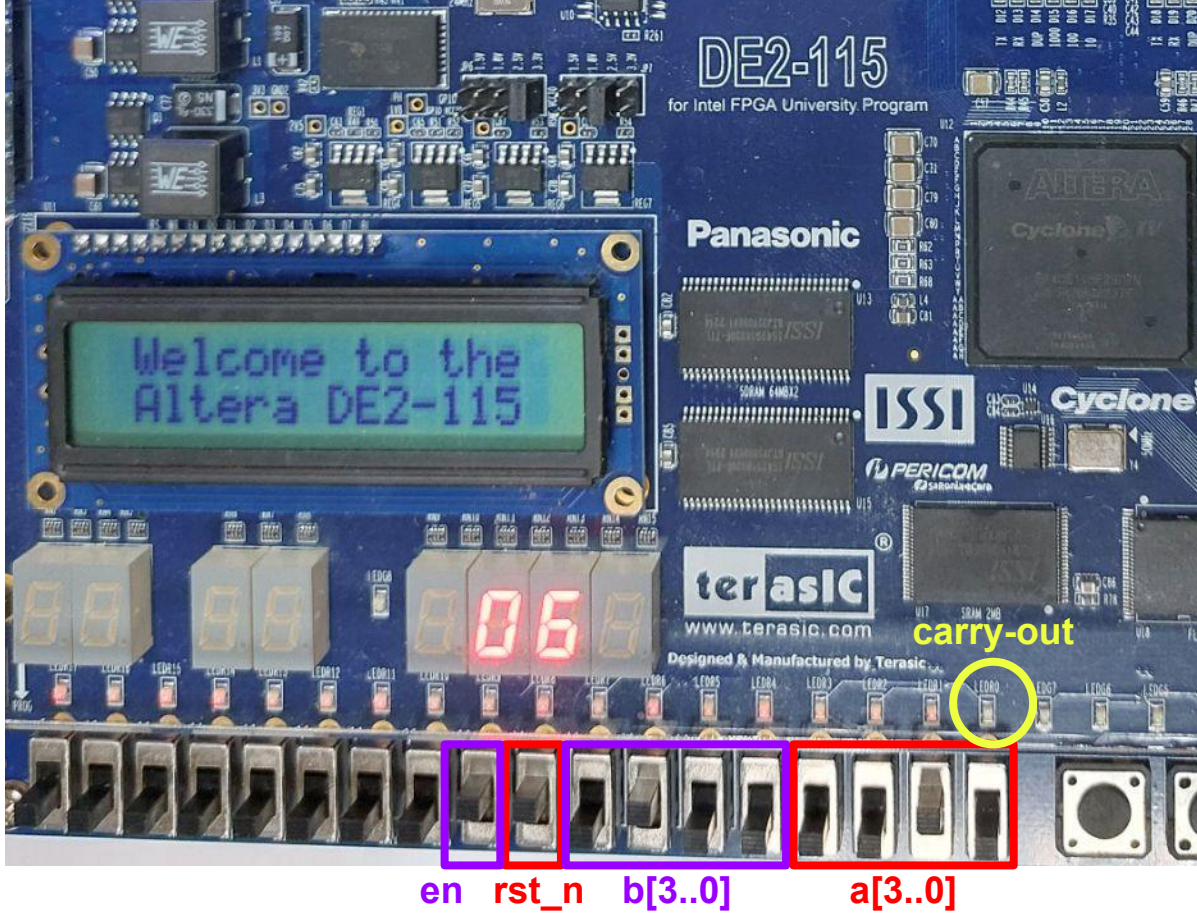
	Resource	Usage
1	Estimated Total logic elements	28
2		
3	Total combinational functions	28
4	Logic element usage by number of LUT inputs	
1	-- 4 input functions	19
2	-- 3 input functions	4
3	-- <=2 input functions	5
5		
6	Logic elements by mode	
1	-- normal mode	25
2	-- arithmetic mode	3
7		
8	Total registers	5
1	-- Dedicated logic registers	5
2	-- I/O registers	0
9		
10	I/O pins	26
11		
12	Embedded Multiplier 9-bit elements	0
13		
14	Maximum fan-out node	rst_n~input
15	Maximum fan-out	13
16	Total fan-out	149
17	Average fan-out	1.75

Analysis & Synthesis Resource Usage Summary

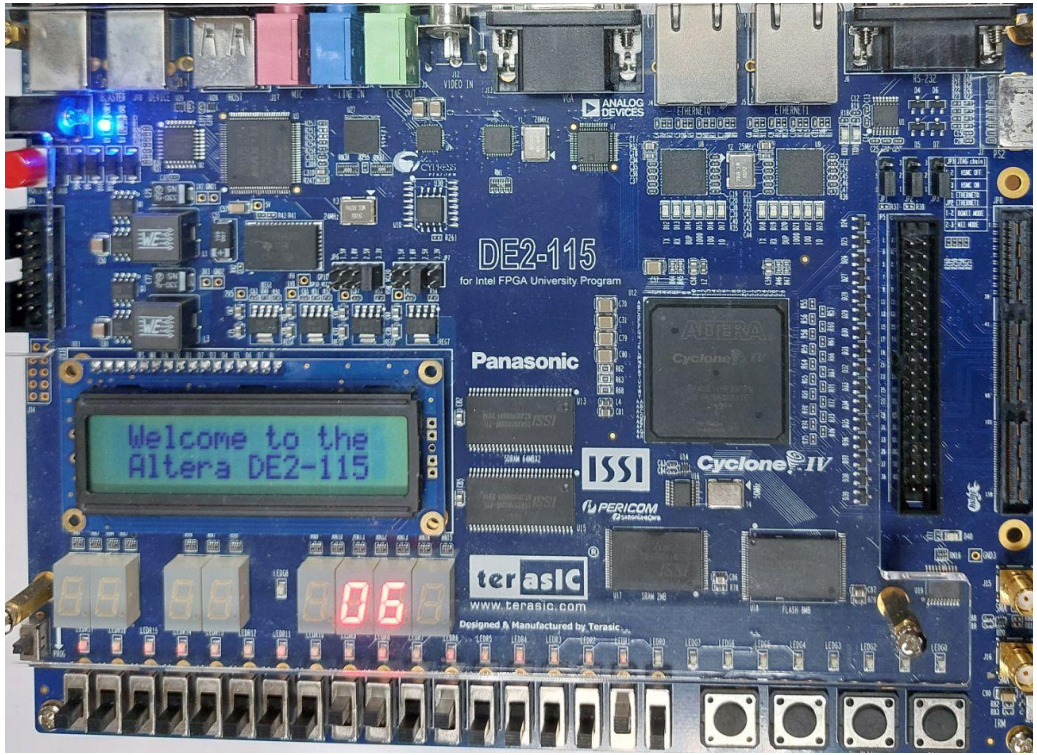
<<Filter>>

	Resource	Usage
1	Estimated Total logic elements	21
2		
3	Total combinational functions	21
4	Logic element usage by number of LUT inputs	
1	-- 4 input functions	15
2	-- 3 input functions	0
3	-- <=2 input functions	6
5		
6	Logic elements by mode	
1	-- normal mode	18
2	-- arithmetic mode	3
7		
8	Total registers	5
1	-- Dedicated logic registers	5
2	-- I/O registers	0
9		
10	I/O pins	26
11		
12	Embedded Multiplier 9-bit elements	0
13		
14	Maximum fan-out node	rst_n~input
15	Maximum fan-out	8
16	Total fan-out	122
17	Average fan-out	1.56

Start my experiment on Altera DE2-115

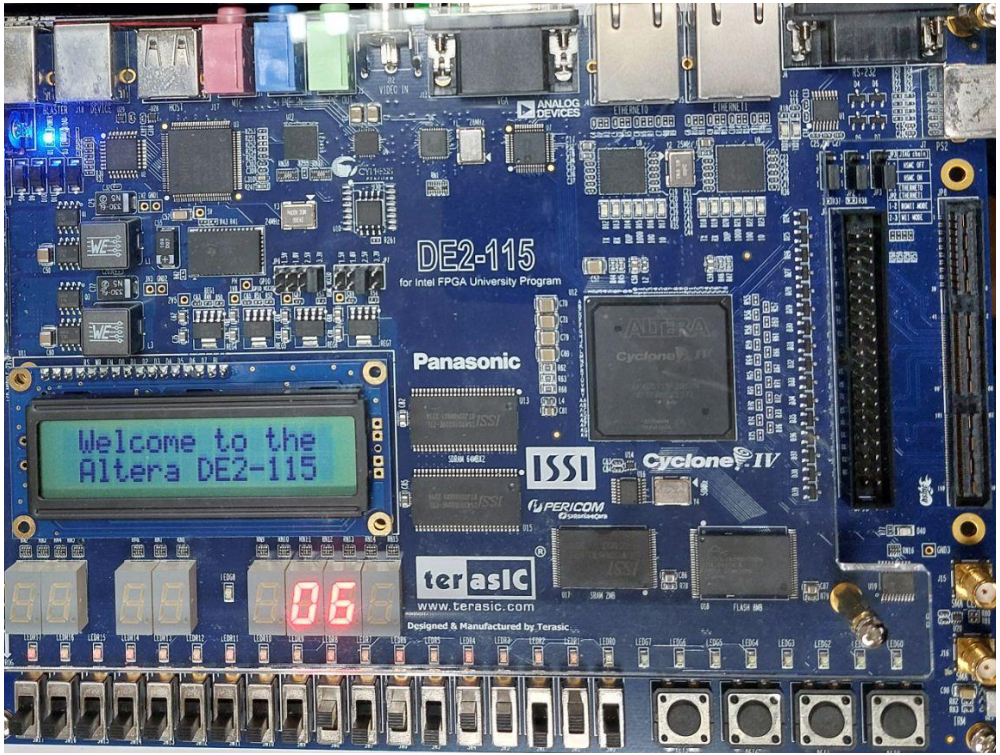


Experiment: 2 + 4 = 6

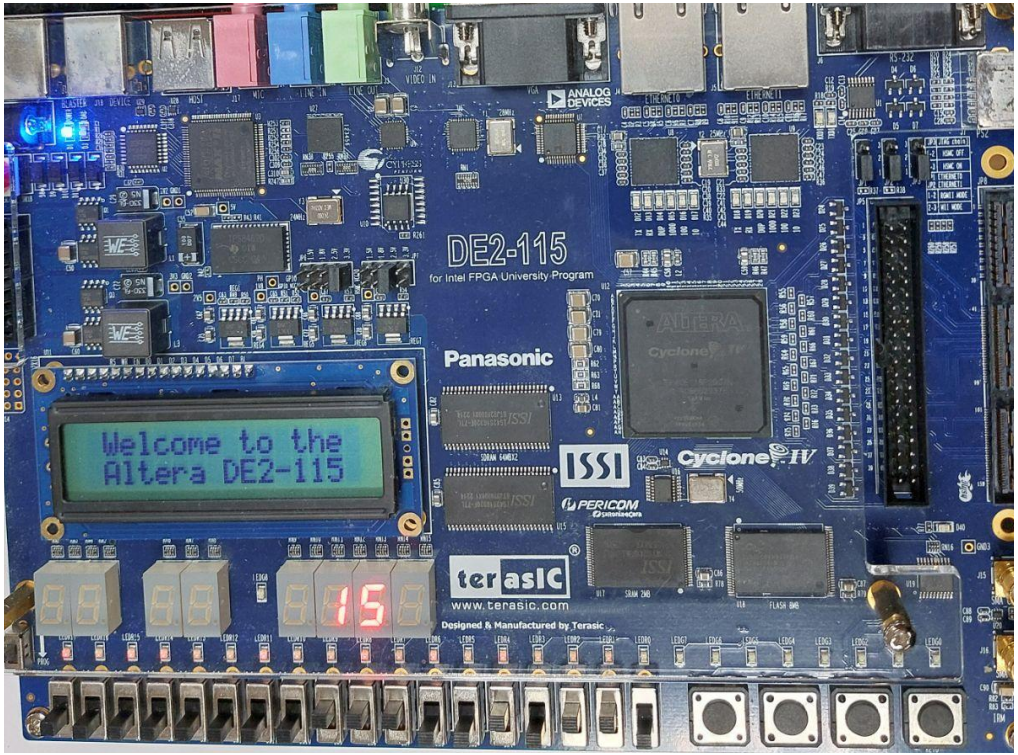


Experiment: disabled.

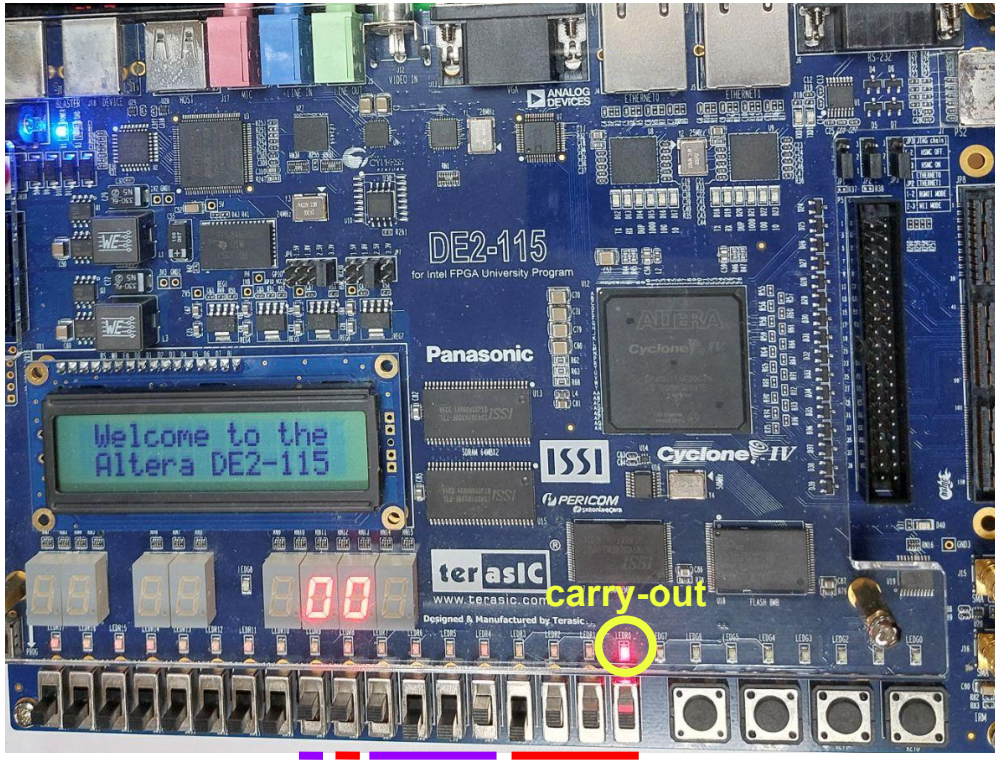
The previous value remains even if the inputs change.



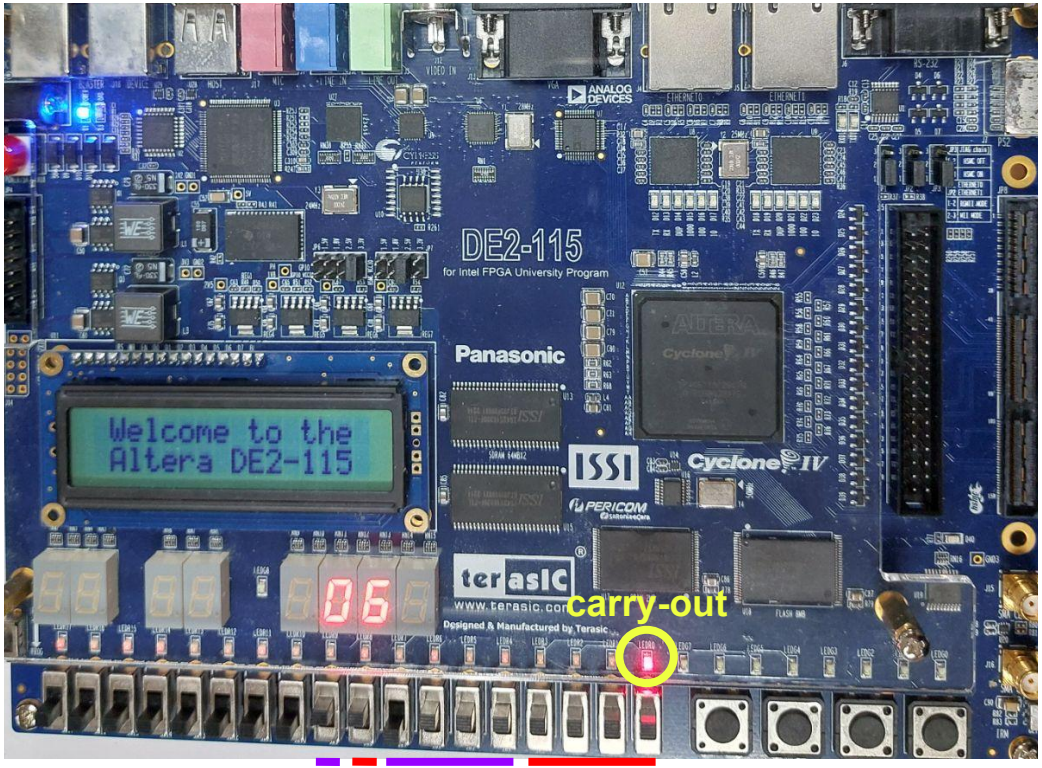
Experiment: 6 + 9 = 15

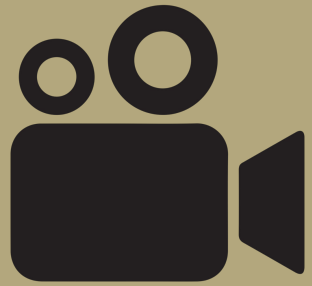


Experiment: 7 + 9 = carry-out



Experiment: 15 + 7 = 6 + carry-out





Experiment Video

4-bit Carry Adder

Conclusion

- ripple-carry adder (RCA)
- carry-lookahead adder (CLA)
- ModelSim & DE2-115

Design trade-offs

Ripple-carry adder (RCA)

- **Strength**: Minimal logic and straightforward implementation make the RCA efficient in resource usage and easy to verify.
- **Limitation**: Carry propagation is sequential, so worst-case propagation delay grows linearly with bit width and becomes the dominant performance bottleneck as operand size increases.

Carry-lookahead adder (CLA)

- **Strength**: Generates carries in parallel using generate/propagate logic, enabling much faster addition for wider bit widths because carry computation can be performed concurrently.
- **Limitation**: Increased complexity and higher utilization of FPGA logic resources; the CLA requires more LUTs and routing, which can complicate timing closure on the target device.

Conclusion

- For small bit widths or resource-constrained designs, the ripple-carry adder (RCA) is a pragmatic, low-overhead choice.
- For high-performance or wide-operand addition where throughput and latency are critical, the carry-lookahead adder (CLA) is preferable despite its larger area and implementation complexity.
- Our combined simulation and hardware results provide consistent evidence supporting these recommendations.

電子與AI應用專業人才養成班第三期

FPGA與數位IC晶片設計

Thanks for your attention

勞動部勞動力發展署 WORKFORCE DEVELOPMENT AGENCY, MINISTRY OF LABOR

NYCU 國立陽明交通大學

NYCU LASER SYSTEM RESEARCH CENTER LSRC 陽明交大雷射系統研究中心